Date: 24/03/2026
Name: Dhanya S

I am Dhanya S from Kakunje Software working as an Intern. Today I had my 42 session it was an interesting session and mam had taught us about connecting the Frontend and Backend. Today I learned about adding a note in the backend using the CRUD operations like delete and update, displaying it in the browser using frontend (reactjs), we can edit and delete the note what we have written and that note is stored in MongoDB compass with the userId and name.

MERNSTACK

1. **backend**

config

db.js

```
import mongoose from "mongoose";
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB Connected");
  } catch (error) {
    console.log(error);
    process.exit(1);
  }
};
export default connectDB;
```

controllers

authController.js

```
import User from "../models/User.js";
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";

//register
export const registerUser = async(req,res)=>{
    try{
        const {name,email,password} = req.body;
//check user exists
        const userExists = await User.findOne({email});
        if(userExists){
```

```
            return res.status(400).json({message:"user already exists"});
        }
//hash password
    const hashedPassword = await bcrypt.hash(password,10);
    const user = new User({
        name,
        email,
        password:hashedPassword
    });
    await user.save();
    res.json({message:"user registered successfully"});
}catch(error){
    res.status(500).json({message:error.message})
}
}

//login
export const loginUser = async(req,res)=>{
    try{
        const {email,password}=req.body;
        const user = await User.findOne({email});
        if(!user){
            return res.status(400).json({message:"user not found"});
        }
        const isMatch = await bcrypt.compare(password,user.password);
        if(!isMatch){
            return res.status(400).json({message:"invalid password"});
        }

//create token
```

```
        const token = jwt.sign(
            {userId:user._id,role:user.role},
            process.env.JWT_SECRET,
            {expiresIn:"1d"}
        );
        res.json({message:"login successful",token});
    }catch(error){
         res.status(500).json({message:error.message})
    }
}
```

noteController.js

```
import Note from "../models/Note.js";

export const addNote = async (req,res)=>{
    const note=await Note.create({
        userId:req.user.id,
        text:req.body.text
    });
    res.json(note);
};

export const getNotes=async (req,res)=>{
    const notes=await Note.find({userId:req.user.id});
    res.json(notes)
};

export const updateNote = async(req,res)=>{
    try{
        const note=await Note.findById(req.params.id);
```

```
        if(!note){
            return res.status(404).json({message:"note not found"});
        }
        note.text=req.body.text || note.text;
        const updatedNote = await note.save();
        res.json(updatedNote);
    }catch(error){
        res.status(500).json({message:error.message});
    }
}

export const deleteNote = async(req,res)=>{
    try{
        const note =await Note.findById(req.params.id);
        if(!note){
            return res.status(404).json({message:"note not found"});
        }
        await note.deleteOne();
        res.json({message:"note deleted"})
    }catch(error){
        res.status(500).json({message:error.message})
    }
}
```

middleware

authMiddleware.js

```
import jwt from "jsonwebtoken";

const authMiddleware = (req,res,next)=>{
    const header = req.header("Authorization");
```

```
    if(!header){
        return res.status(401).json({message:"no token"});
    }
    try{
        const token = header.split(" ")[1];
        const verified = jwt.verify(token,process.env.JWT_SECRET);
        req.user = verified;
        next();
    }catch(error){
        res.status(400).json({message:"invalid token"})
    }
}

export default authMiddleware;
```

models

Note.js

```
import mongoose from "mongoose";

const noteSchema = new mongoose.Schema({
    userId:{
        type:mongoose.Schema.Types.ObjectId,
        ref:"User"
    },
    text:String
},{timestamps:true});
export default mongoose.model("Note",noteSchema);
```

User.js

```
import mongoose from "mongoose";
```

```
const userSchema = new mongoose.Schema({
   name:{
      type:String
   },
   email:{
      type:String,
      unique:true
   },
   password:{
      type:String
   }
});

export default mongoose.model("User",userSchema);
```

routes

authRoutes.js

```
import express from "express";
import { registerUser,loginUser } from "../controllers/authController.js";
import authMiddleware from "../middleware/authMiddleware.js";

const router = express.Router();
router.post("/register",registerUser);
router.post("/login",loginUser);
router.get("/profile",authMiddleware,(req,res)=>{
   res.json({message:"welcome",user:req.user});
});

export default router;
```

noteRoutes.js

```
import express from "express";
import { addNote,deleteNote,getNotes, updateNote } from "../controllers/noteController.js";
import authMiddleware from "../middleware/authMiddleware.js";

const router = express.Router();
router.post("/",authMiddleware,addNote);
router.get("/",authMiddleware,getNotes);
router.put("/:id",authMiddleware,updateNote);
router.delete("/:id",authMiddleware,deleteNote);

export default router;
```

.env

```
MONGO_URI = mongodb://localhost:27017/userlogin
JWT_SECRET = mysecretkey
```

package.json

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "keywords": [],
```

```
  "author": "",
  "license": "ISC",
  "type": "module",
  "dependencies": {
    "bcrypt": "^6.0.0",
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.6",
    "dotenv": "^17.3.1",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.3.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

server.js

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import connectDB from "./config/db.js";
import authRoutes from "./routes/authRoutes.js";
import noteRoutes from "./routes/noteRoutes.js";

dotenv.config();
connectDB();
const app = express();
app.use(cors());
app.use(express.json());
```

```
app.use("/api/auth",authRoutes);
app.use("/api/notes",noteRoutes);
app.listen(5000,()=>{
console.log("server is running on port 5000");
})
```

2. **frontend**

src

pages

Dashboard.jsx

```
import React, { useState, useEffect } from "react";
import API, { updateNote, deleteNote } from "../services/api";
import "../styles/Dashboard.css";

const Dashboard = () => {
  const [notes, setNotes] = useState([]);
  const [text, setText] = useState("");
  const [editId, setEditId] = useState(null);
  const fetchNotes = async () => {
    try {
      const res = await API.get("/notes");
      setNotes(res.data);
    } catch (err) {
      console.error(err);
    }
  };

  useEffect(() => {
    fetchNotes();
  }, []);

  const addNote = async () => {
    try {
      if (editId) {
        await updateNote(editId, { text });
        setEditId(null);
```

```
      } else {
        await API.post("/notes", { text });
      }
      setText("");
      fetchNotes();
    } catch (err) {
      console.error(err);
    }
};

const handleDelete = async (id) => {
    try {
      await deleteNote(id);
      fetchNotes();
    } catch (err) {
      console.error(err);
    }
};

const logout = () => {
    localStorage.removeItem("token");
    window.location.href = "/";
};
return (
    <div className="container">
      <h2>Dashboard</h2>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Enter note"
```

```
        />
        <button onClick={addNote}>
          {editId ? "Update" : "Add"}
        </button>
        <button onClick={logout}>Logout</button>
        <ul>
          {notes.length > 0 ? (
            notes.map((n) => (
              <li key={n._id}>
                <span>{n.text}</span>

                <button
                  onClick={() => {
                    setText(n.text);
                    setEditId(n._id);
                  }}>
                  Edit
                </button>

                <button onClick={() => handleDelete(n._id)}>
                  Delete
                </button>
              </li>
            ))
          ) : (
            <p>No notes found</p>
          )}
        </ul>
      </div>
    );
```

```
};

export default Dashboard;
```

Login.jsx

```
import React from 'react'
import { useState } from 'react';
import "../styles/Login.css";
import {useNavigate} from "react-router-dom"
import API from "../services/api";

const Login = () => {
   const[form,setForm]=useState({
      email:"",
      password:""
   });

  const navigate = useNavigate();

   const handleSubmit=async(e)=>{
      e.preventDefault();
      const res = await API.post("/auth/login",form);
      localStorage.setItem("token",res.data.token)
      navigate("/dashboard");
      alert("login successful");
   }
 return (
   <form className="login-form" onSubmit={handleSubmit}>
   <h2>Login form</h2>
```

```
    <input type="text" placeholder='enter email'
    onChange={(e)=>setForm({...form,email:e.target.value})} />

    <input type="text" placeholder='enter password'
    onChange={(e)=>setForm({...form,password:e.target.value})} />

    <button>Login</button>
    </form>
  )
}

export default Login
```

Register.jsx

```
import React from 'react'
import { useState } from 'react';
import "../styles/Register.css";
import API from "../services/api";
import { useNavigate } from 'react-router-dom';

const Register = () => {
    const [form,setForm]=useState({
    name:"",
    email:"",
    password:""
});

const navigate = useNavigate();

const handleSubmit=async(e)=>{
```

```
    e.preventDefault();
    await API.post("/auth/register",form);
    navigate("/");
    alert("registered successfully");
}

  return (
    <form className="register-form" onSubmit={handleSubmit}>
    <h2>Register form</h2>

    <input type="text" placeholder='enter name'
    onChange={(e)=>setForm({...form,name:e.target.value})} />

    <input type="text" placeholder='enter email'
    onChange={(e)=>setForm({...form,email:e.target.value})} />

    <input type="text" placeholder='enter password'
    onChange={(e)=>setForm({...form,password:e.target.value})} />

    <button>Register</button>
   </form>
  )
}

export default Register
```

services

api.jsx

```
import axios from "axios";
```

```
const API = axios.create({
   baseURL: "http://localhost:5000/api"
});

API.interceptors.request.use((req) => {
   const token = localStorage.getItem("token");
   if (token) {
      req.headers.Authorization = `Bearer ${token}`;
   }
   return req;
});

export const updateNote = (id, data) => API.put(`/notes/${id}`, data);
export const deleteNote = (id) => API.delete(`/notes/${id}`);

export default API;
```

authServices.jsx

```
import axios from "axios";

const API = axios.create({
   baseURL: "http://localhost:5000/api/auth"
});

export const register =(data)=>API.post("/register",data);
export const login =(data)=>API.post("/login",data);
```

styles

Dashboard.css

```
.container {
```

```
  max-width: 500px;
  margin: 50px auto;
  padding: 25px;
  background: pink;
  border-radius: 12px;
  box-shadow: 0 4px 15px rgba(0,0,0,0.1);
  text-align: center;
  font-family: Arial, sans-serif;
}
.container h2 {
  margin-bottom: 20px;
  color: black;
}
.container input {
  width: 70%;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 6px;
  outline: none;
  margin-right: 10px;
}
.container input:focus {
  border-color: #4CAF50;
}
.container button {
  padding: 8px 12px;
  margin: 5px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
```

```
  font-size: 14px;
}
.container button:first-of-type {
  background-color: #4CAF50;
  color: white;
}
.container button:first-of-type:hover {
  background-color: green;
}
.container button:nth-of-type(2) {
  background-color: #f44336;
  color: white;
}
.container button:nth-of-type(2):hover {
  background-color: #d32f2f;
}
ul {
  list-style: none;
  padding: 0;
  margin-top: 20px;
}
li {
  background: #f9f9f9;
  padding: 12px;
  margin-bottom: 10px;
  border-radius: 8px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

```
li span {
  flex: 1;
  text-align: left;
}
```

Login.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f4f6f8;
}
.login-form {
  width: 300px;
  margin: 100px auto;
  padding: 25px;
  background: pink;
  border-radius: 10px;
  text-align: center;
}
.login-form h2 {
  margin-bottom: 20px;
  color: black;
}
.login-form input {
  width: 94%;
  padding: 10px;
  margin: 10px 0;
  border: 1px solid #ccc;
  border-radius: 6px;
  outline: none;
```

```
}
.login-form input:focus {
  border-color: #4CAF50;
}
.login-form button {
  width: 100%;
  padding: 10px;
  margin-top: 10px;
  background: #4CAF50;
  color: black;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 15px;
}
.login-form button:hover {
  background: green;
  color: white;
}
```

Register.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f4f6f8;
}
.register-form {
  width: 320px;
  margin: 80px auto;
  padding: 25px;
```

```
  background: rgb(237, 153, 167);
  border-radius: 10px;
  text-align: center;
}
.register-form h2 {
  margin-bottom: 20px;
  color: black;
}
.register-form input {
  width: 94%;
  padding: 10px;
  margin: 10px 0;
  border: 1px solid #ccc;
  border-radius: 6px;
  outline: none;
}
.register-form input:focus {
  border-color: #2196F3;
}
.register-form button {
  width: 100%;
  padding: 10px;
  margin-top: 10px;
  background: #2196F3;
  color: black;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}
.register-form button:hover {
```

```
  background: darkblue;
  color: white;
}
```

App.jsx

```
import React from 'react'
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Dashboard from './pages/Dashboard';
import Login from './pages/login';
import Register from './pages/register';

const App = () => {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/dashboard" element={<Dashboard />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App;
```

main.jsx

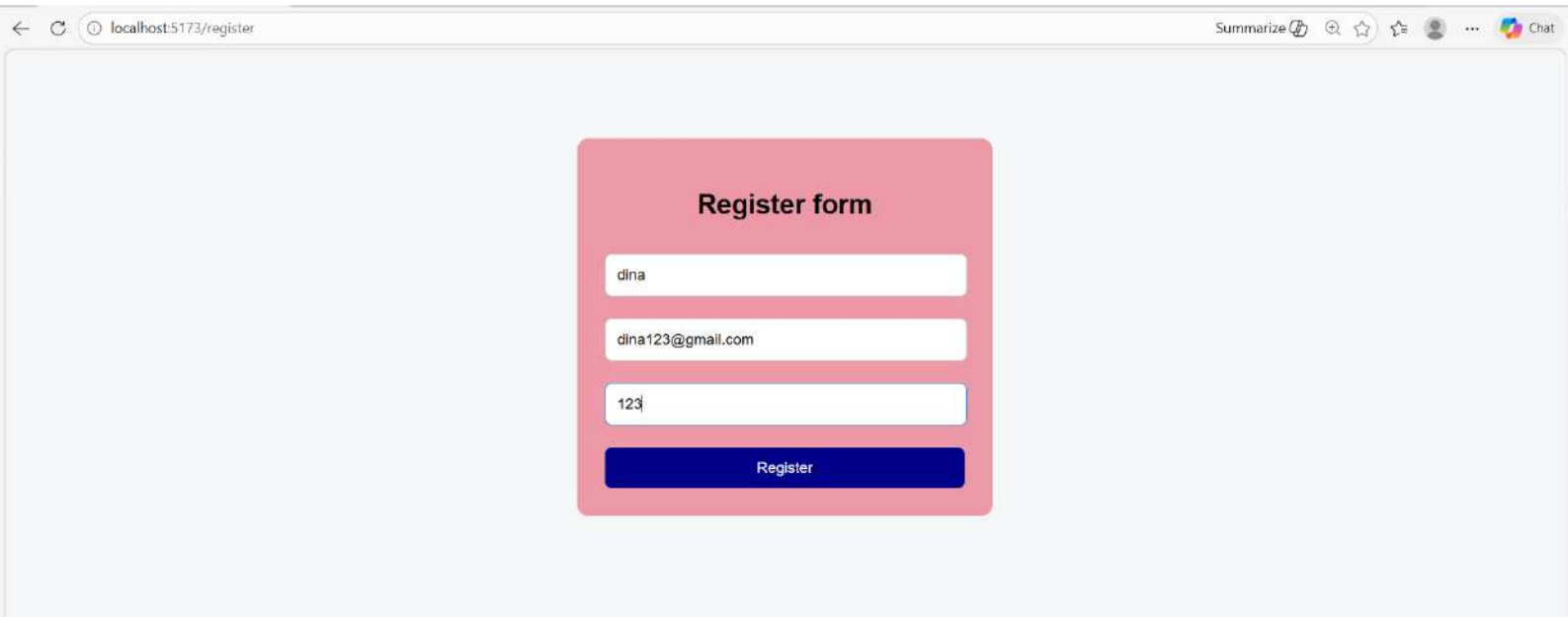
```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
```

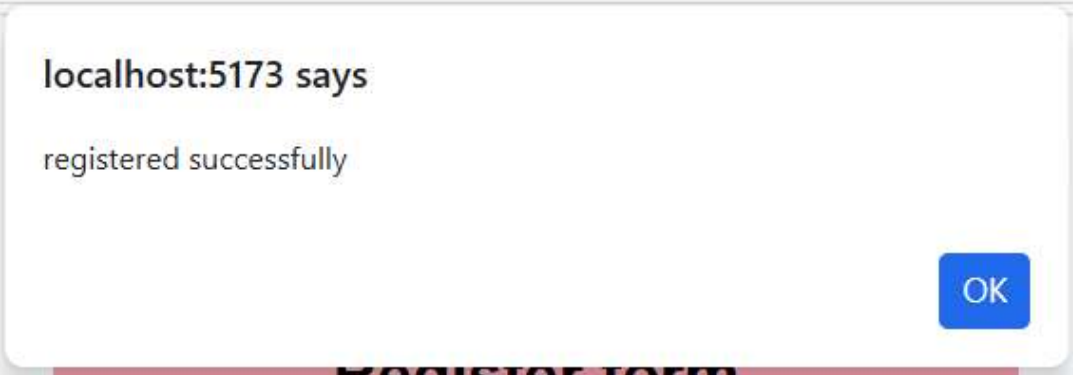
```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

package.json

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.13.6",
    "react": "^19.2.4",
    "react-dom": "^19.2.4",
    "react-router-dom": "^7.13.1"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.4",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
```

```
    "eslint": "^9.39.4",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.4.0",
    "vite": "^8.0.1"
  }
}
```


localhost:5173/register
Summarize
Chat
Register form
dina
dina123@gmail.com
123
Register



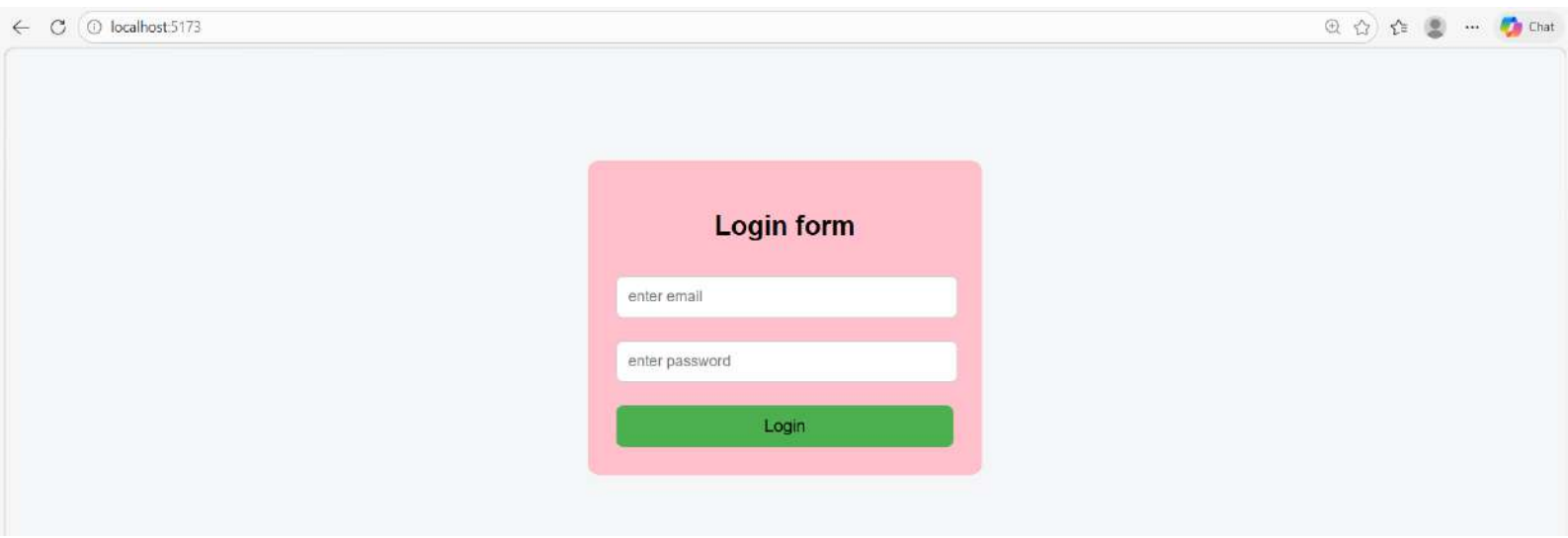
Register form

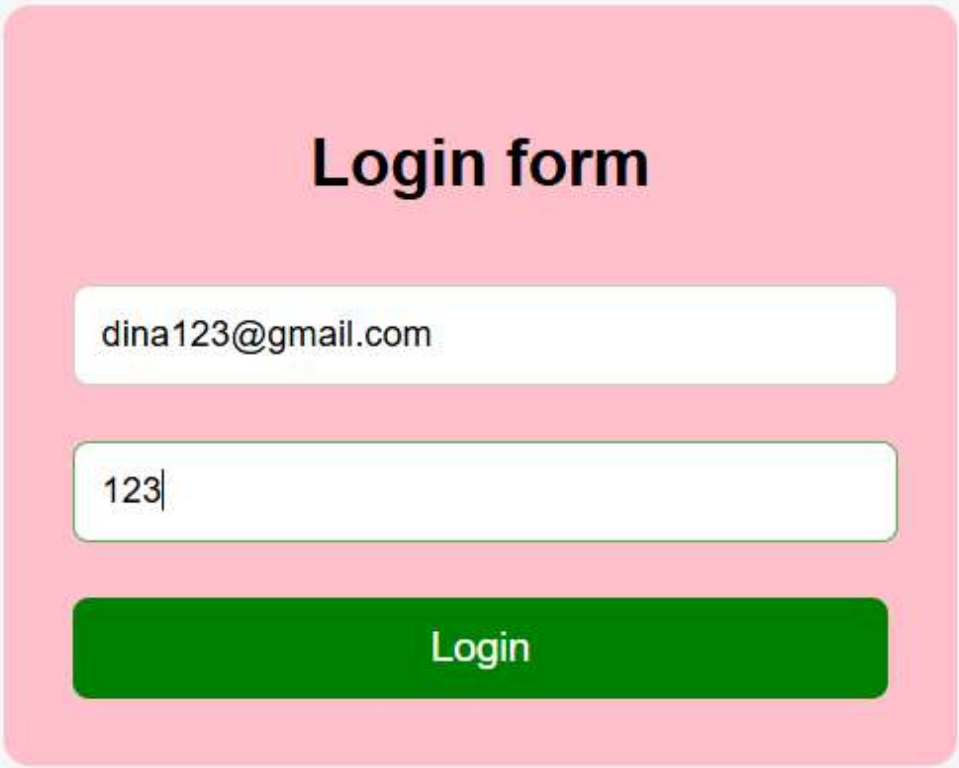
dina

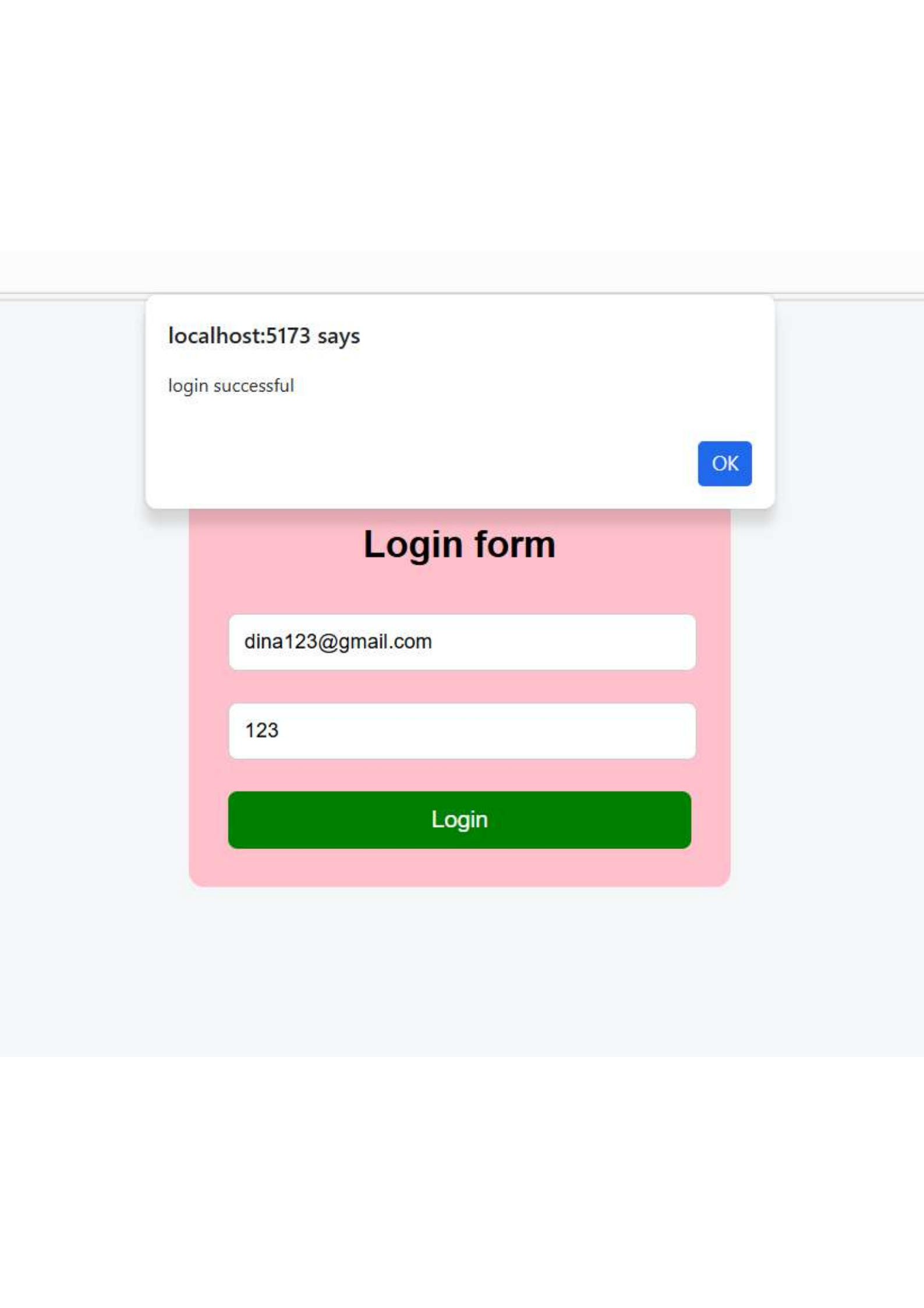
dina123@gmail.com

123

Register


localhost:5173
Chat
Login form
enter email
enter password
Login


Login form
dina123@gmail.com
123
Login


localhost:5173 says
login successful
OK
Login form
dina123@gmail.com
123
Login

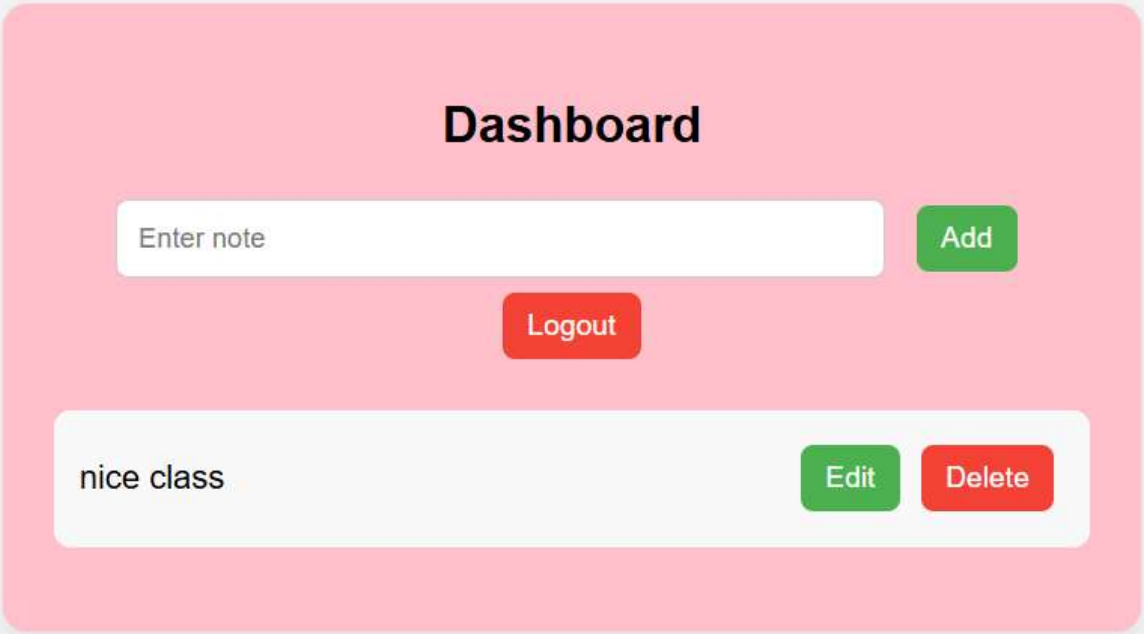
Dashboard

Enter note

Add

Logout

No notes found


Dashboard
Enter note
Add
Logout
nice class
Edit
Delete

Dashboard

nice classsss

Update

Logout

nice class

Edit

Delete

Dashboard

Enter note

Add

Logout

nice classsss

Edit

Delete

Dashboard

Enter note

Add

Logout

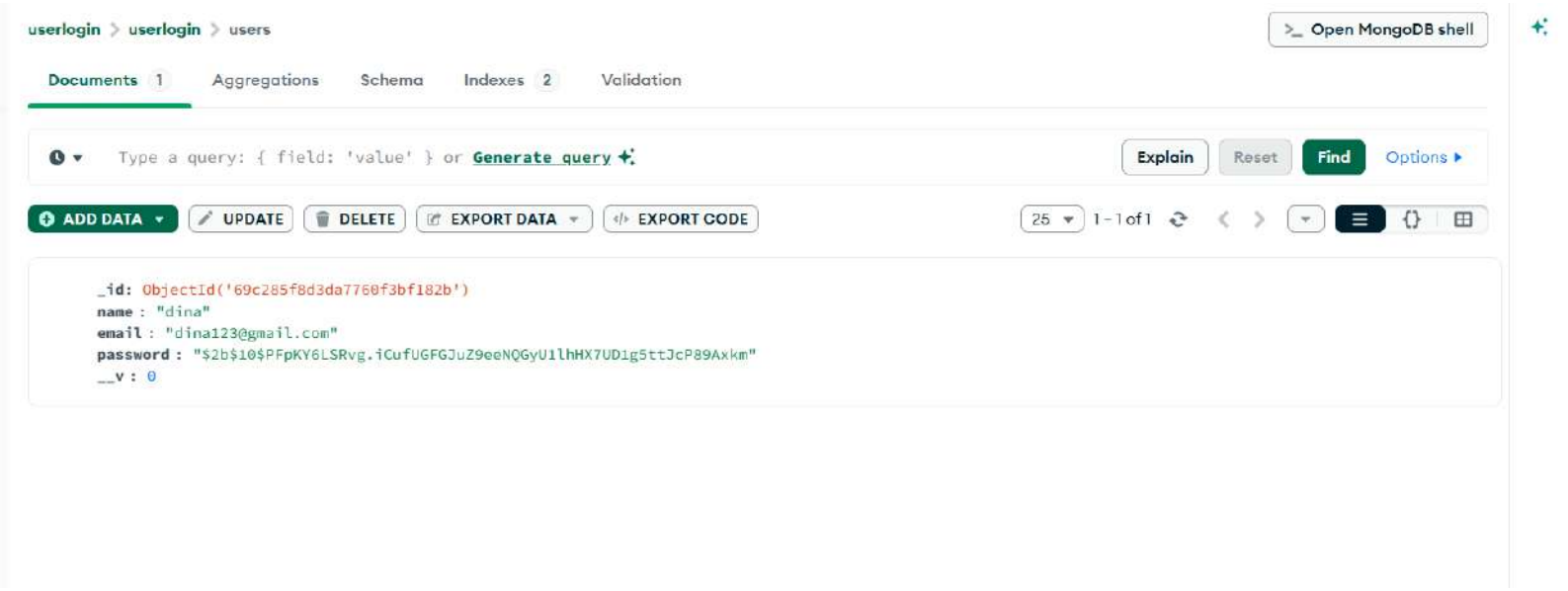
No notes found

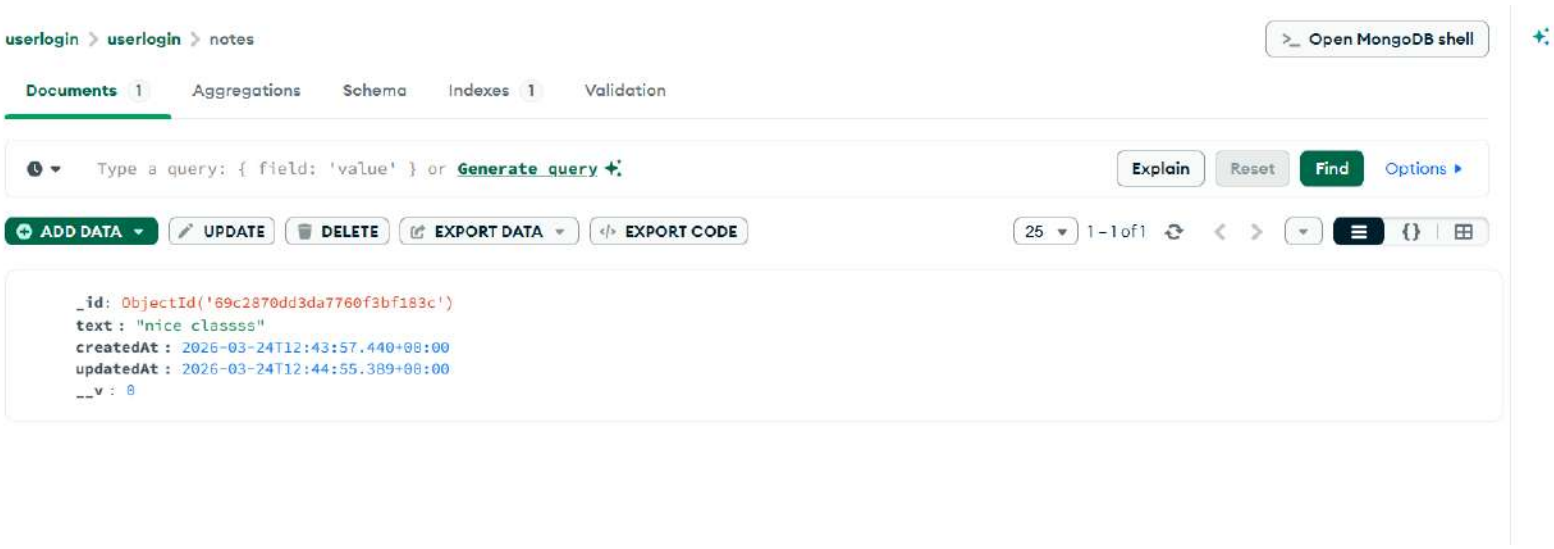
Login form

enter email

enter password

Login


userlogin > userlogin > users
Open MongoDB shell
Documents 1
Aggregations
Schema
Indexes 2
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
UPDATE
DELETE
EXPORT DATA
EXPORT CODE
25
1 - 1 of 1
_id: ObjectId('69c285f8d3da7760f3bf182b')
name : "dina"
email : "dina123@gmail.com"
password : "$2b$10$PFpKY6LSRvg.iCufUGFGJuZ9eeNQGyU1lhHX7UD1g5ttJcP89Axkm"
__v : 0


userlogin > userlogin > notes
Open MongoDB shell
Documents 1
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
UPDATE
DELETE
EXPORT DATA
EXPORT CODE
25
1 - 1 of 1
_id: ObjectId('69c2870dd3da7760f3bf183c')
text : "nice classss"
createdAt : 2026-03-24T12:43:57.440+08:00
updatedAt : 2026-03-24T12:44:55.389+08:00
__v : 0